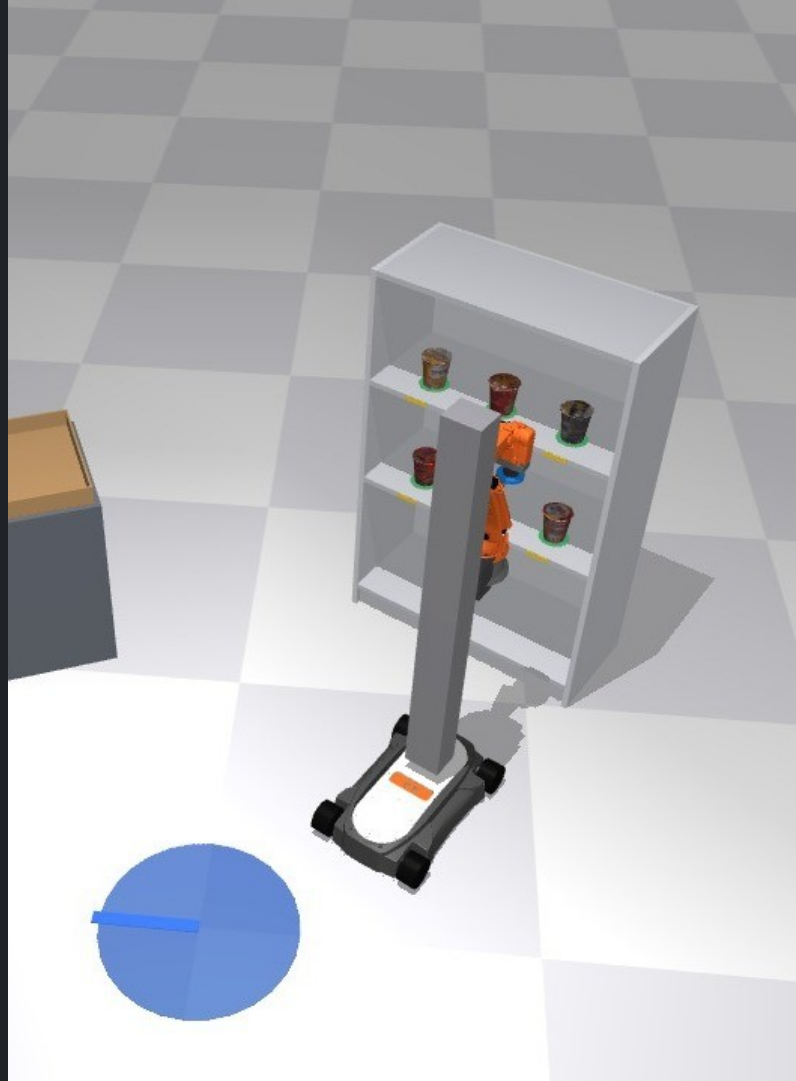
Behavior Tree 기반 컵라면 진열

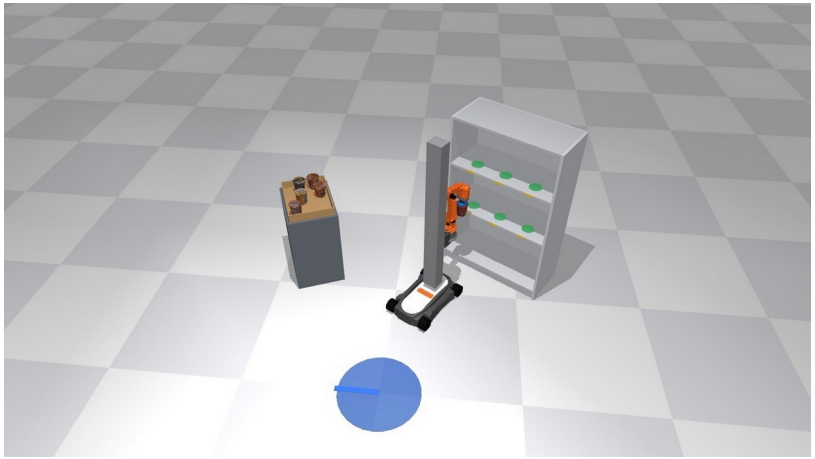
BehaviorOS 확장 · MuJoCo · 개조 youBot

이하영 · Robotics Deployment 사전 과제

github.com/hayoung0Lee/cupramen



과제와 결과 한눈에



무작위로 놓인 상자에서 컵라면 6 개를 집어
2 단 선반의 6 칸에 하나씩 진열

행동 트리 JSON 5 단계 (b2 → b6) · 노드 15 개 · 테스트 54 개

6/6 × 5

seed 0~4 모두 6 개 진열

254~295 초 , 로봇–가구 접촉 0 건

5 회 완주

실패를 일부러 일으켜도

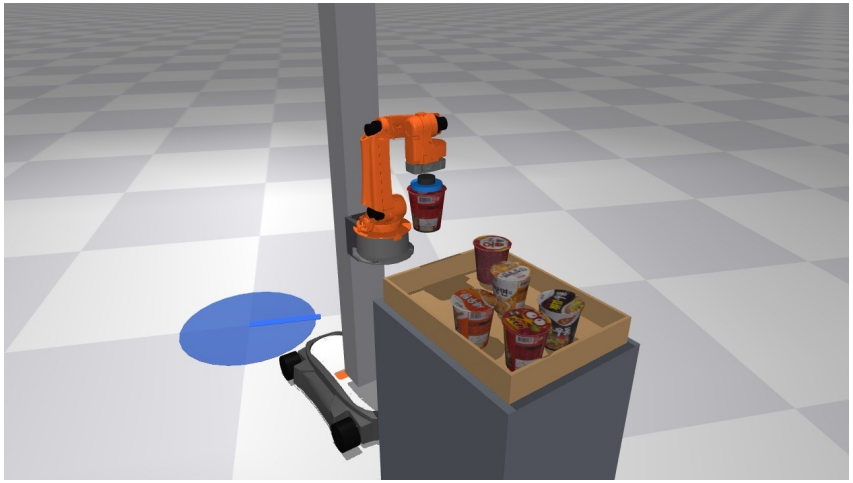
흡착 실패 · 놓침에서 복구 , 바닥에 떨어진 컵만 포기

2 단계

사람이 넘겨받아 내려놓기

돌려주면 로봇이 검사하고 이어서

로봇 : 과제에 맞게 개조한 youBot



로봇 모델이 지정되지 않음 (임바디먼트 자유) → 익숙한 youBot 을 바탕으로

1. **리프트 기둥** — 어깨 0.70~1.30 m — 위 칸 1 m 에 닿게
2. **팔 링크 연장** — 링크 0.23 · 0.20 m 로 — 선반 깊이까지
3. **흡착 툴** — 그리퍼 대신 — 뚜껑을 위에서 잡게
4. **베이스 단순화** — 관절 3 개로 직접 이동 — 바퀴 물리 대신 , 계산 안정

흡착 모델링 : 뚜껑에 닿았나 + 중심 2.5 cm + 기울기 8° 안이면 흡착판과 컵을 weld(켜고 끄는 고정 연결) 로 붙임

시스템 구조

trees/*.json	무엇을 어떤 순서로 (행동 트리)
BehaviorOS	트리 실행 엔진 — 이번에 확장
shelving/	트리 노드 : go_to_box, pick_cup, go_to_shelf, place_cup, recover, takeover …
shelving/utils/	노드가 같이 쓰는 도우미 : 움직임 · 팔 계산 (IK) · 길찾기 (A*)
sim/	MuJoCo 세상 : 제공 장면 + 로봇 , 흡착

tick 한 번 = 20 ms

① 트리가 실행 중인 노드를 찾음
② 노드가 모터 명령을 한 조각 씀
③ 물리가 2 ms × 10 번 계산 → 로봇이 실제로 움직임

devices

객체
로봇 · IK
길찾기 · 조종자

blackboard

데이터
지금 컵 · 칸 상태
실패 기록

기존 BehaviorOS 대비 수정

모두 실제로 겪은 문제 → 작은 트리로 재현하는 테스트를 먼저 쓰고 수정 · 원본 : git 태그 behavioros-v0.1-legacy

1 모르는 노드 타입 → 즉시 오류

오타 난 노드가 조용히 ' 성공 ' 으로 바뀌어 단계를 건너뜀

2 노드 시작마다 reset()

재시도한 노드가 지난 상태에서 이어져 바로 끝남

3 retry · repeat_until · inverter

JSON 으로 재시도 · 반복을 쓸 수 없었음 (repeat_until 은 신규)

4 fail(이유) → 실패 기록

FAILURE 만으로는 왜인지 몰라 복구할 수 없음

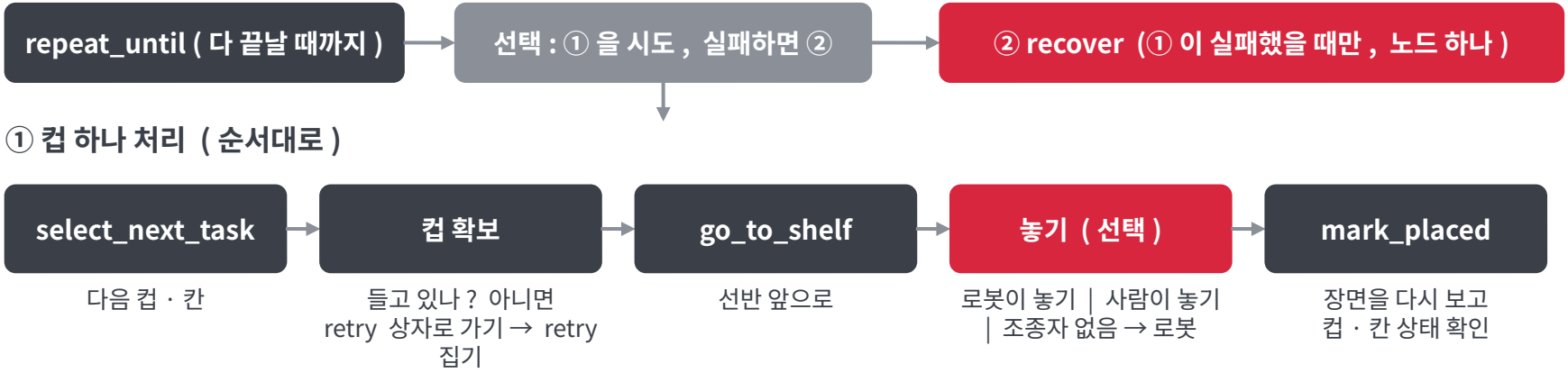
5 시뮬레이션 시계 주입

실제 시계라 실행 방식마다 소요 시간이 달라짐

6 devices 분리

로봇 같은 객체가 데이터용 블랙보드에 섞임

행동 트리 설계와 의도



1 **노드 = 사람이 읽는 단위**

' 상자로 가기 ',' 컵 집기 '. 팔을 내리고 · 누르고 · 확인하는 세부 단계는 노드 안의 단계 목록

2 **그 노드만 쓰는 계산은 그 노드 파일에**

예 : 상자 면 고르기는 go_to_box.py, 선반 자리 고르기는 go_to_shelf.py

3 **컵 · 칸 상태는 장면을 보고 판정**

계획대로 고치지 않음 → 실패 뒤에도 믿을 수 있는 상태에서 다시 계획

시연 1 — 1 단계 자율 진열



seed 0, 처음부터 끝까지 (4 배속) · 오른쪽 아래 : 팔 끝 클로즈업

```
mjpython scripts/run_bt.py trees/b6_takeover.json --seed 0
```

영상 링크 (Google Drive) ↗

실패 복구 : 실패 크기에 따라 되돌아가는 정도를 다르게

retry — 그 자리에서 다시

흡착 실패 , 컵이 상자 안에서 위치가 바뀜
→ 다시 집을 때마다 컵의 지금 위치로 다시 계산

recover — 정리하고 다시 계획

운반 중 놓침 , 재시도 소진 , 갈 자리 없음
① 안전 정리 : 상자 위면 위로 , 선반 앞이면 뒤로
② 시도 횟수 +1 (같은 컵 3 번이면 포기)
③ 장면을 다시 보고 컵 · 칸 상태 판정 → 다음 회차가 처음부터

모든 실패는 이유를 남김 : seal_failed · cup_dropped · blocked · no_shelf_spot …

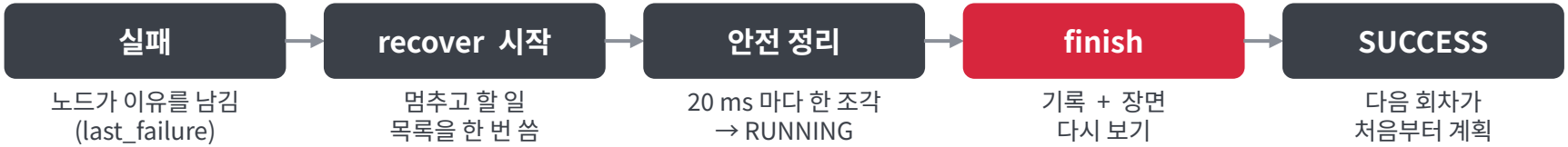
사례 : 바닥에 떨어진 컵

문제 선반 앞 바닥 컵에 차체가 걸려 시간 초과를 반복

고침 바닥 컵을 길찾기 지도에 장애물로 등록 , 3 초 막히면 바로 감지 , 못 가는 칸은 기억하고 다른 칸

발견 고치기 전 좋은 결과는 로봇이 컵을 밀고 지나간 것이었음 → 검사 도구가 바닥 컵 접촉도 세게 고침

recover 안에서 : 할 일 목록을 쓰고 조금씩 진행



할 일 목록 (지금 로봇 위치에 따라 달라짐)

① 도구가 상자 위 → 똑바로 위로 (컵 사이를 쓸고 지나가지 않게)

② 상자 · 선반 근처 → 몸통을 똑바로 뒤로 (선반 칸에서 도구를 빼냄)

③ 팔을 운반 자세로 접음

④ finish: 시도 횟수 +1, 못 간 칸은 ' 막힘 ', 장면 다시 보기

움직임은 한 tick 에 조금씩 , 끝나면 목록에서 지움 . 목록이 비면 SUCCESS (실패로 끝나지 않으니 반복이 계속됨)

장면을 다시 보고 판정 → 다음 회차가 정해짐

흡착판에 그대로	→ 들고 있는 채로 이어서 운반
상자로 다시 떨어짐	→ 지금 놓인 위치에서 다시 집기
바닥에 떨어짐	→ 팔이 안 닿음 → 포기
칸 옆에 어긋나게 놓임	→ 그 칸은 막힘 , 다른 칸으로

같은 컵 3 번 실패 → 건너뜀 (끝없이 반복하지 않게)

시연 2 — 실패 복구



흡착 실패 → 다시 · 상자로 떨어짐 → 새 위치로 · 바닥에 떨어짐 → 정리하고 계속

```
mjpython scripts/run_bt.py trees/b5_recovery.json --seed 0 --inject seal-fail:1,drop-lift:3,drop-carry:4
```

영상 링크 (Google Drive) ↗

사람 개입 : 진열의 마지막 단계에서만

과제 : " 정확한 위치에 진열할 수 있도록 , 진열의 마지막 단계에서 … 제어권을 넘겨받을 수 있는 구조 "



통과 → 이어서 진열 · 벗어남 → 이유를 말하며 다시 요청 (최대 3 번)

언제 넘어가나

- 사람이 t 를 누를 때
- 로봇이 정확한 진열을 약속 못 할 때 : 컵이 흡착판에 4° 넘게 기울거나 2 cm 넘게 비켜 붙음 · 놓을 자세 없음 · 놓은 뒤 검사 실패

사람이 할 수 있는 것

- 손끝 앞뒤 · 좌우 · 위아래 1 cm / 2 mm
- 베이스 이동 · 회전 , 흡착 잡기 / 놓기
- 화면에 칸까지 남은 오차 (mm) 가 계속 표시됨

안전장치

- 사람 명령도 팔 · 차체가 가구 3 mm 안이면 거부
- 조종자가 없으면 로봇이 직접 놓음
- 상자에서 집기 · 운반 중에는 넘기지 않음

시연 3 — 사람 개입



t → 칸 앞에서 넘겨받기 → 오차 보며 내려놓기 → h → 로봇이 검사하고 이어서 (키 입력 재생)

```
mjpython scripts/run_bt.py trees/b6_takeover.json --seed 0 --operator "script:t,f,f,F,F,F,F,space,r,r,r,h"
```

영상 링크 (Google Drive) ↗

반복 실험 5 회 (seed 0~4)

seed	진열	시간	이동 거리	직선 기준	개입	실패
0	**6/6**	271.0 s	23.23 m	22.88 m	0	0
1	**6/6**	270.6 s	22.18 m	19.95 m	0	0
2	**6/6**	273.6 s	26.26 m	19.54 m	0	0
3	**6/6**	295.0 s	34.44 m	41.01 m	0	0
4	**6/6**	254.4 s	14.33 m	16.45 m	0	0

배치 : 제공 randomize.py · 진열 판정 : count_placed · 거리 : MobilityLog · 시간은 시뮬레이션 시간

실패를 일부러 일으킨 5 회

`seal:0.3 · drop-lift:3 · drop-carry:2`

5 회 완주

진열 5/6 × 4 회 , 4/6 × 1 회
바닥에 떨어진 컵은 팔이 닿지 않아 포기
가구 · 바닥 컵 접촉 0



seed 0 완료

한계와 다음 단계

인식	시뮬레이션의 정답 위치 사용 (과제 : 인식 방법 자유)	→	컵 상태 판정 (observe) 만 카메라 인식으로 바꾸면 트리는 그대로
바닥에 떨어진 컵	팔이 바닥에 닿지 않아 포기	→	사람 요청 대상으로 확장
노드 단위	' 컵 집기 ' 처럼 굵은 단위 , 세부 동작은 노드 안	→	세부 동작을 노드로 노출 → LLM 이 트리를 조합 (BehaviorOS 원래 목표)
다음 컵 고르기	로봇에서 직선거리로 가장 가까운 컵	→	실제로 설 상자 면 기준으로

AI 도구 활용

1 활용 도구

- Claude Code (Anthropic), VS Code 에서 사용
- (추가 도구가 있으면 여기에)

2 사용 목적

- 과제 분석 · 계획 문서
- MuJoCo 셋업 , 로봇 모델 (MJCF) 과 메시 변환
- IK· 길찾기 · 노드 ·BehaviorOS 수정 구현
- 디버깅 , 개념 설명 (학습)

3 결과 검증

- 수정마다 실패하는 테스트를 먼저 → 통과 (54 개)
- 구조를 바꿀 때마다 5 회 결과 CSV 가 그대로인지 비교
- 접촉 · 도달 검사 , 새 환경에서 README 대로 설치
- 직접 화면으로 확인 → 실제 버그 2 개 발견
- 설계 결정 (범위 , 구조) 은 근거를 보고 내가 결정

구현 속도가 빨라진 만큼 검증을 중요하게 : 자동 검사 + 직접 화면 확인 + 결정은 사람이

부록 : 디버깅 사례와 수치 근거

weld 가 1 m 옆에 붙음

코드로 만든 weld 의 기준점 기본값이 (0,1,0) → 데이터 구조를 실험으로 확인해 0 으로

충돌 검사가 들고 있는 컵을 안 봄

검사용 복사본에서 컵을 흡착판에 붙여 같이 옮겨 검사

계획한 팔 자세 ≠ 실제 자세

리프트 + 팔은 같은 손끝 위치를 여러 자세로 만듦 → 검사 때 실제 동작을 그대로 재생

보이는 모양 > 충돌 모양

팔 받침이 상자에 박혀 보임 → 메시로 충돌 판정 , 간격 6 cm

수치 근거

- 제어 주기 20 ms (물리 2 ms × 10)
- 뚜껑 위 대기 3 cm, 들기 15 cm (컵 11 cm)
- 흡착 중심 2.5 cm · 기울기 8°
- 사람 요청 4° · 2 cm
- 진열 판정 15 mm · 5°
- 막힘 감지 3 초
- 같은 컵 포기 3 번